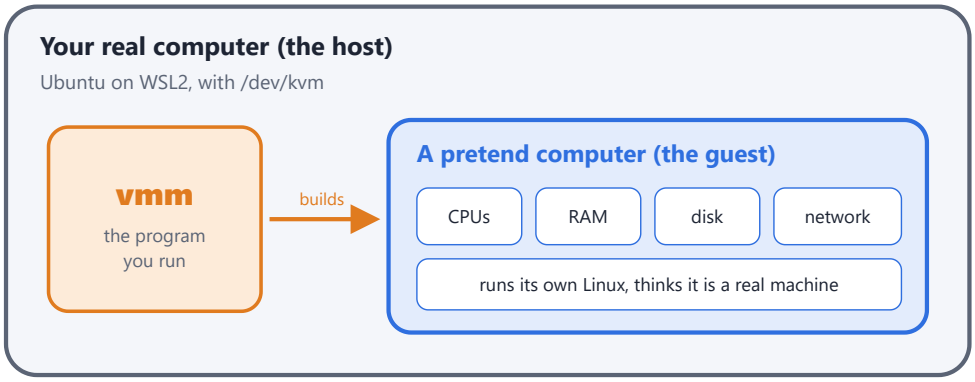
KVM-HELLO · USER MANUAL

vmm: a tiny computer inside your computer

How to use the VMM: every command, what to type, and what you will see. Organised milestone by milestone, from M1 (print one line) to M8 (prove the code is safe).

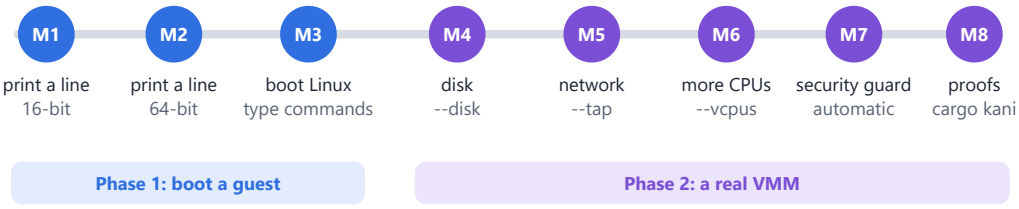


Binary: `target/debug/vmm` · Covers Phase 1 (M1–M3) and Phase 2 (M4–M8) · Written for the code as of commit `385d68d`

What's inside

1. What is a VMM, and what is it for?
2. Before you start: one-time setup
3. How to read the commands in this manual
4. **After M1**: run a tiny 16-bit program
5. **After M2**: run a tiny 64-bit program
6. **After M3**: boot real Linux and type into it
7. **After M4**: give the guest a hard disk
8. **After M5**: give the guest a network cable
9. **After M6**: give the guest more CPUs
10. **After M7**: the security guard (seccomp)
11. **After M8**: proving the code is safe
12. Everything at once
13. Exit codes
14. When something goes wrong
15. Cheat sheet (one page)

> Each milestone chapter only shows what that milestone **added**. Everything from earlier chapters still works afterwards. So the manual grows like the program did.



Start

What is a VMM, and what is it for?

> A **VMM** (virtual machine monitor) is a program that makes a **pretend computer**. Software running inside it (the **guest**) believes it has its own CPU, memory, disk and network. It doesn't: the VMM fakes all of them using your real computer (the **host**).

Who does the real work?



What people use VMMs like this for

Use	Plain words
Serverless / cloud functions	AWS Lambda runs every customer's code inside its own tiny VM, made by **Firecracker**, the VMM this project imitates. Thousands of strangers share one machine and can't see each other.
Running untrusted code safely	If the code inside is evil, it is trapped in the pretend computer. A VM wall is much stronger than a container wall.
Fast, small machines	A "microVM" has only the devices it needs (serial, disk, network), so it starts in well under a second and uses little memory.
Testing kernels and OSes	Boot a new Linux kernel without risking your real machine.
Learning (this repo)	See exactly how a computer boots, how devices work and how hypervisors are secured, by building one.

TIPThis VMM is a **learning project**. It runs as a normal user (never root), has one serial console, at most one disk, one network card and four CPUs. It is not meant to replace Firecracker.

Setup

Before you start: one-time setup

Do these once. Commands starting with `sudo` need your password.

1. **Check KVM is there and you may use it.**

```
$ ls -l /dev/kvm           # the device must exist
$ groups | grep -w kvm      # you must be in the kvm group
```

2. **Install the host tools** (assembler, kernel build tools, disk tools, network tools).

```
$ sudo apt install nasm busybox-static cpio build-essential flex bison bc \
     libelf-dev libssl-dev e2fsprogs python3 iproute2 iputils-ping strace
```

3. **Build the VMM.** Run from the repo folder `~/hypervisor/kvm-hello`.

```
$ cargo build
    Finished `dev` profile [unoptimized + debuginfo] target(s)
```

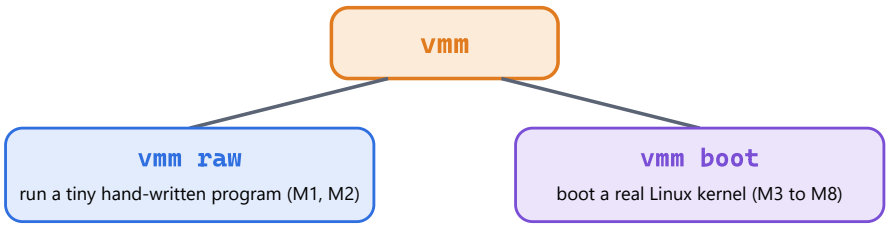
The program is now at `target/debug/vmm`. Every command in this manual starts with that path.

4. **Ask the VMM what it can do.**

```
$ target/debug/vmm --help
Usage: vmm <COMMAND>

Commands:
  raw   Run a flat binary guest loaded at guest-physical 0x1000
  boot  Boot a Linux bzImage with a serial console on stdin/stdout
  help  Print this message or the help of the given subcommand(s)
```

The two commands



Ask for the details of either one with `target/debug/vmm raw --help` or `target/debug/vmm boot --help`. `vmm --version` prints the version.

Read me

How to read the commands in this manual

Syntax boxes

```
SYNTAX
vmm raw --image <FILE> --mode <real|long> [--mem-mib <N>]
```

You see	It means
`--image`	A **flag**. Type it exactly like that, with two dashes.
`<FILE>`	Replace with your own value. Don't type the `< >`.
`<real|long>`	Pick **one** of these words: `real` or `long`.
`[--mem-mib <N>]`	Optional. Leave it out and the default is used. Don't type the `[ ]`.
`--disk-ro` (no value)	An on/off switch. Writing it turns the feature on.

Flags can come in any order after `raw` or `boot`.

Two different prompts: who are you talking to?

> Once Linux boots (M3 onward), your keyboard is plugged into the **guest**. The prompt tells you where you are.

HOST (your Ubuntu)

$

cargo, scripts, sudo, ping from host

GUEST (Linux inside vmm)

phase1#

mount, ip, ping from guest, reboot -f

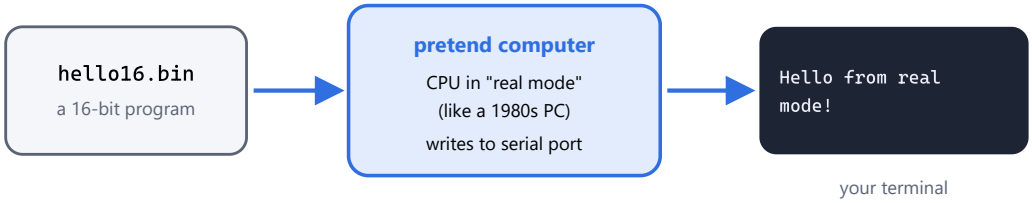
In the examples, `$` lines are typed on the **host**, `phase1#` lines are typed in the **guest**, grey text is what the computer prints, and `# ...` is a comment: don't type it.

TIPThe guest prompt is called `phase1#` in every milestone, even in Phase 2. That's on purpose: the guest's start-up script never changed, so old tests keep working.

After M1

Run a tiny 16-bit program

> The very first thing the VMM could do: load a 30-byte program into the pretend computer, let it print **one line**, and stop.



Step by step

1. **Turn the assembly source into a program file** (once, or after editing a `.asm`):

```
$ scripts/build-guests.sh
built guests/build/hello16.bin
built guests/build/hello64.bin
```

2. **Run it:**

```
$ target/debug/vmm raw --image guests/build/hello16.bin --mode real
Hello from real mode!
$ echo $?            # shows the exit code of the last command
0
```

SYNTAX

```
vmm raw --image <FILE.bin> --mode real [--mem-mib <N>]
```

Flag	Default	What it does
`--image <FILE>`	required	The program to run. A raw "flat" binary, 1 byte to 24 KiB. It is copied to address `0x1000` of the guest's memory and the CPU starts there.
`--mode real`	required	Start the CPU in 16-bit **real mode**, the mode every PC wakes up in.
`--mem-mib <N>`	`16`	How much RAM the pretend computer gets, in MiB. Allowed: 1 to 3072.

What happens when it ends

The program finishes with the `hlt` ("halt") instruction. The VMM sees that and exits with code **0**. If the program crashes the CPU instead, the VMM exits with code **1**.

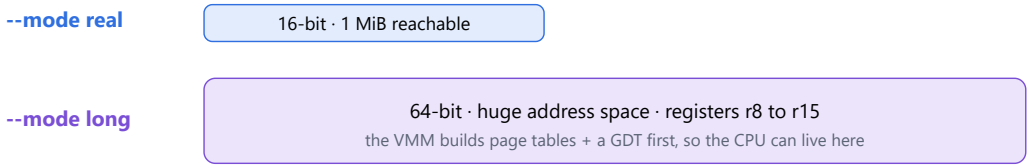
TIP **Make it say something else.** Open `guests/hello16.asm`, change the text in the line `msg: db "Hello from real mode!", 10, 0`, then run `scripts/build-guests.sh` and the `vmm raw` command again.

WATCH OUT `raw` mode never reads your keyboard. It only prints what the program writes to the serial port (I/O port `0x3f8`).

After M2

Run a tiny 64-bit program

> Same idea as M1, but now the VMM switches the CPU into **64-bit long mode** before the program starts: the mode modern Linux lives in.



You only change one word on the command line. The VMM does the hard setup for you.

Step by step

```
$ scripts/build-guests.sh            # if you haven't already
$ target/debug/vmm raw --image guests/build/hello64.bin --mode long
Hello from long mode!
```

SYNTAX

```
vmm raw --image <FILE.bin> --mode <real|long> [--mem-mib <N>]
```

NEW IN THIS MILESTONE

`--mode long` : start the CPU in 64-bit long mode, with the first 1 GiB of memory mapped and a stack ready. Instruction pointer is `0x1000` , stack pointer is `0x8FF0` , interrupts off.

WATCH OUT**The mode must match the program.** `hello64.bin` with `--mode real` (or the reverse) runs the bytes the wrong way. You'll get nothing, garbage, or exit code 1. It can't harm your computer, it just won't work.

Which mode do I pick?

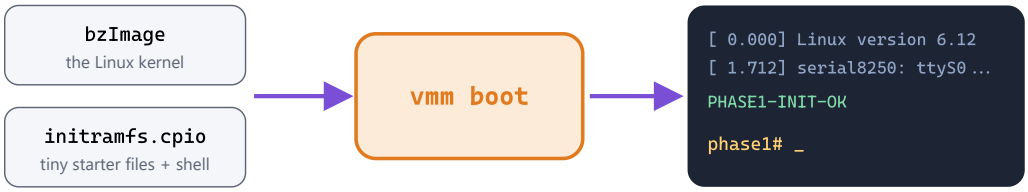
Your `.asm` file starts with	Use
`bits 16`	`--mode real`
`bits 64`	`--mode long`

Both programs are loaded at address `0x1000` , so each `.asm` file starts with `org 0x1000` .

After M3

Boot real Linux and type into it

> The big one. `vmm boot` loads a real Linux kernel, Linux starts up inside the pretend computer, and you get a **shell you can type into**.



Step 1: build the kernel and starter files (once)

```
$ scripts/build-kernel.sh       # downloads + builds Linux 6.12 → artifacts/bzImage (slow: minutes)
$ scripts/build-initramfs.sh    # packs BusyBox + /init → artifacts/initramfs.cpio (fast)
```

Both are safe to run again: the kernel is only rebuilt if its version or config changed.

Step 2: boot

```
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio
[    0.000000] Linux version 6.12.111 ...
   ... lots of start-up messages ...
PHASE1-INIT-OK

BusyBox v1.36.1 built-in shell (ash)
phase1#
```

`PHASE1-INIT-OK` means Linux started and handed over to the shell. You are now **inside the guest**.

Step 3: type commands in the guest

```
phase1# uname -r                # which kernel is running?
6.12.111
phase1# echo vmm-$((6*7))         # proves your keystrokes reach the guest
vmm-42
phase1# ls /                    # look around: the guest's own files
phase1# cat /proc/meminfo | head -1
```

Step 4: leave

```
phase1# reboot -f               # the ONLY clean way out
$ echo $?
0
```

WATCH OUT**Ctrl-C does not quit the VMM.** While the guest runs, every key goes to the guest, including Ctrl-C (it stops the guest's current command, like on a normal Linux). To leave, type `reboot -f`. If the guest is truly stuck, from a **second** terminal run `pkill vmm`; your terminal is restored automatically.

After M3

vmm boot: the flags (Phase 1)

SYNTAX

```
vmm boot --kernel <bzImage> [--initrd <FILE.cpio>] [--mem-mib <N>] [--cmdline "<TEXT>"]
```

Flag	Default	What it does
`--kernel <FILE>`	required	The Linux kernel to boot, in **bzImage** format (what `make` produces). Use `artifacts/bzImage`.
`--initrd <FILE>`	none	The starter filesystem (a cpio archive) that holds the shell and `/init`. Use `artifacts/initramfs.cpio`. Leave it out and Linux has nothing to run: it panics, reboots, and the VMM exits 0.
`--mem-mib <N>`	`512`	Guest RAM in MiB. Allowed: 128 to 3072.
`--cmdline "<TEXT>"`	`console=ttyS0 reboot=k panic=1 pci=off acpi=off`	Settings passed to the Linux kernel. Max 2047 characters. Put it in quotes. If you change it, keep `console=ttyS0` or you won't see any output.

Examples

```
# more memory for the guest
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio --mem-mib 1024

# a quieter boot: add "quiet" to the kernel settings
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio \
      --cmdline "console=ttyS0 reboot=k panic=1 pci=off acpi=off quiet"

# see what the VMM itself is doing (logs go to stderr, not mixed with guest output)
$ RUST_LOG=debug target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio
2>vmm.log
```

Where things go

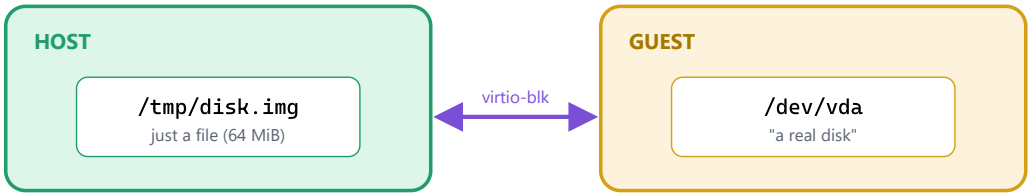
Stream	Carries
Your screen (stdout)	Only what the guest prints on its serial console.
Your keyboard (stdin)	Fed into the guest's serial console, key by key.
stderr	Messages from the VMM itself: errors and logs. Default level `warn`; change with `RUST_LOG=info` or `RUST_LOG=debug`.

TIPRunning the tests from this phase: `cargo test` runs M1/M2. `cargo test -- --ignored` also runs the Linux-boot tests (they need the built kernel). `scripts/verify-phase-1.sh` checks every Phase 1 rule and prints PASS/FAIL for each.

After M4

Give the guest a hard disk

> Until M3, everything the guest saved disappeared at reboot (it lived in RAM). M4 adds `--disk` : **a normal file on your host becomes a hard drive in the guest**. What the guest saves stays saved.



Step 1: make a disk (once)

```
$ scripts/build-disk.sh                  # makes artifacts/disk.ext4 (64 MiB, contains
/hello.txt)
$ cp artifacts/disk.ext4 /tmp/disk.img   # work on a COPY, keep the original clean
```

Step 2: boot with the disk

```
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio \
      --disk /tmp/disk.img
```

Step 3: use it in the guest

```
phase1# cat /sys/block/vda/size              # size in 512-byte sectors
131072
phase1# mkdir -p /mnt                        # make a folder to attach the disk to
phase1# mount /dev/vda /mnt                  # attach ("mount") the disk there
phase1# cat /mnt/hello.txt
phase2-disk
phase1# echo hi > /mnt/note.txt             # save something new
phase1# sync                                 # push it to the disk now
phase1# umount /mnt                          # detach before leaving
phase1# reboot -f
```

Step 4: check from the host that it really saved

```
$ debugfs -R "cat /note.txt" /tmp/disk.img
hi
```

Boot again with the same `--disk /tmp/disk.img` and `/mnt/note.txt` is still there.

After M4

Disk flags

SYNTAX (added to vmm boot)

```
[--disk <FILE>] [--disk-ro]
```

Flag	Default	What it does
`--disk <FILE>`	no disk	Attach this file as the guest's disk `/dev/vda`. The file's size must be a non-zero multiple of 512 bytes. It is opened for reading **and writing**.
`--disk-ro`	off	Attach the disk **read-only**. The guest can read it but every write fails, and the host file is never changed. Needs `--disk`.

Read-only example

```
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio \
      --disk /tmp/disk.img --disk-ro
phase1# mkdir -p /mnt && mount /dev/vda /mnt     # Linux notices: mounts read-only
phase1# echo x > /mnt/x.txt
/bin/sh: can't create /mnt/x.txt: Read-only file system
```

NEW IN THIS MILESTONE

Behind the scenes the VMM tells Linux "there is a virtio disk at address `0xd0000000`, interrupt 5" by adding `virtio_mmio.device=4K@0xd0000000:5` to the kernel settings. You never type that yourself.

WATCH OUT**Never give the same disk file to two VMMs at once**, and always `umount` before `reboot -f`. Otherwise the filesystem can be damaged, exactly like pulling the plug on a real PC.

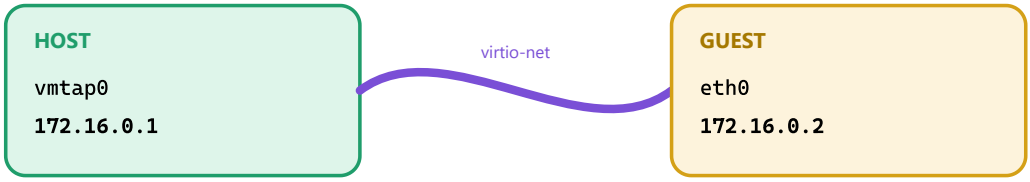
Put your own files on a disk

```
$ mkdir /tmp/stuff && echo "my file" > /tmp/stuff/a.txt
$ truncate -s 64M /tmp/my.img               # make an empty 64 MiB file
$ mkfs.ext4 -q -d /tmp/stuff /tmp/my.img    # format it and copy /tmp/stuff in (no sudo needed)
```

After M5

Give the guest a network cable

> `--tap` plugs a **virtual network cable** between your host and the guest. The host is `172.16.0.1`, the guest is `172.16.0.2`. They can ping each other and send files.



Step 1: create the cable on the host (needs sudo)

```
$ sudo scripts/setup-tap.sh
vmtap0 ready for jyothiradithyams with host address 172.16.0.1/24
```

WATCH OUTRun this again after every `wsl --shutdown` or Windows restart: the TAP disappears then. The VMM itself never needs sudo.

Step 2: boot with the network

```
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio \
      --tap vmtap0
```

Step 3: switch on the guest's network

```
phase1# ip addr add 172.16.0.2/24 dev eth0    # give the guest its address
phase1# ip link set eth0 up                   # switch the card on
phase1# ping -c 3 172.16.0.1                  # talk to the host
3 packets transmitted, 3 packets received, 0% packet loss
```

The guest forgets this at every boot, so type these two `ip` lines each time.

Step 4: try it from the host side (second terminal)

```
$ ping -c 3 172.16.0.2                    # host → guest

# send a file to the guest: serve a folder on the host...
$ cd /tmp && python3 -m http.server 8000 --bind 172.16.0.1
```

```
# ...and download it in the guest
phase1# wget -O /tmp/f http://172.16.0.1:8000/somefile
phase1# sha256sum /tmp/f                    # compare with sha256sum on the host
```

After M5

Network flags

SYNTAX (added to vmm boot)

```
[--tap <IFNAME>] [--guest-mac <aa:bb:cc:dd:ee:ff>]
```

Flag	Default	What it does
`--tap <IFNAME>`	no network	Name of an **existing** TAP interface on the host (normally `vmtap0`). It becomes `eth0` in the guest. The VMM opens it, never creates it.
`--guest-mac <MAC>`	`06:00:ac:10:00:02`	The guest card's hardware address: 6 hex pairs with colons. Must not be a multicast address (first byte odd). Needs `--tap`.

```
# custom MAC
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio \
      --tap vmtap0 --guest-mac 06:00:ac:10:00:09
phase1# ip link show eth0
    link/ether 06:00:ac:10:00:09 ...
```

NEW IN THIS MILESTONE

The network card lives at address `0xd0001000`, interrupt 6 (`virtio_mmio.device=4K@0xd0001000:6`, added for you). The disk keeps its own slot, so you can use `--disk` and `--tap` together.

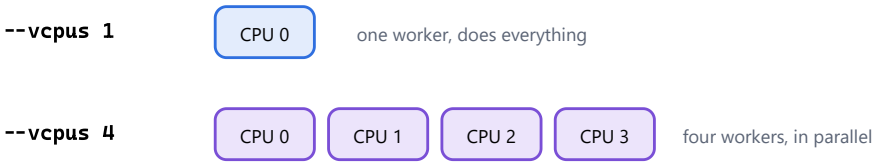
WATCH OUTOnly **one** VMM can use `vmtap0` at a time. A second one fails with an error.

TIPUnder the hood, every packet the guest sends has a 12-byte "virtio header" in front. The VMM wipes it to zeros before handing the packet to the host, so a misbehaving guest can't trick the host's network code.

After M6

Give the guest more CPUs

`--vcpus 4` gives the guest **four CPUs** instead of one. Each one is a separate thread on your host, and Linux in the guest uses them all.




```
$ target/debug/vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio \
      --vcpus 4
phase1# nproc                              # how many CPUs does Linux see?
4
phase1# grep -c ^processor /proc/cpuinfo   # same question, another way
4
phase1# grep LOC /proc/interrupts          # timer ticks per CPU: all four non-zero = all working
```

SYNTAX (added to vmm boot)

```
[--vcpus <1-4>]
```

Flag	Default	What it does
`--vcpus <N>`	`1`	Number of virtual CPUs, 1 to 4. `0` or `5` is refused (exit 2).

NEW IN THIS MILESTONE

The VMM writes an **MP table** into guest memory: a little "parts list" that tells Linux how many CPUs exist and how interrupts are wired. Only CPU 0 starts running; Linux wakes up the others itself. On `reboot -f`, the VMM stops and waits for **all** CPU threads before exiting.

TIPMore vCPUs don't make boot faster on this WSL2 machine (nested virtualization is slow). They let the guest run several programs at the same time.

After M7

The security guard (seccomp)

> From M7 on, each part of the VMM has a **guard with a short list** of things it is allowed to ask the host kernel for. Anything not on the list? The guard **stops the whole VMM instantly**. You don't type anything: it's always on.



Why?

The guest is untrusted. If an attacker inside the guest ever found a bug in the VMM's device code and took it over, the guard means they still can't open your files, start programs or make network connections on the host.

SYNTAX (added to vmm boot)

```
[--no-seccomp]
```

Flag	Default	What it does
`--no-seccomp`	off (guard on)	Turn the guard **off**. For debugging only. Prints a warning: `vmm: warning: seccomp disabled (--no-seccomp); for debugging only`

If the guard ever fires

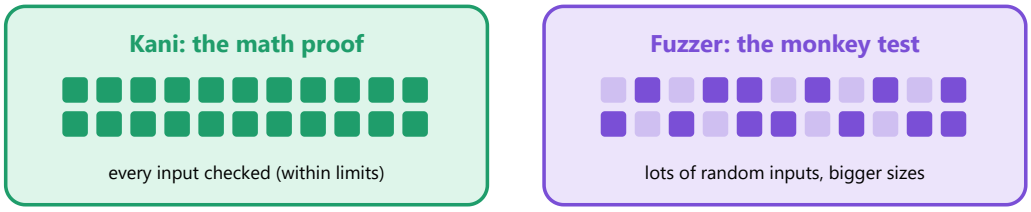
```
$ target/debug/vmm boot ...
Bad system call        # the shell's message for "killed by SIGSYS"
$ reset                  # your terminal may look broken: type this (even blind) + Enter
```

WATCH OUTA seccomp kill can't be caught, so the VMM can't put your terminal back to normal. Typing `reset` fixes it. If this happens in normal use, it's a VMM bug: run again with `--no-seccomp` to confirm, and report which syscall was missing.

After M8

Proving the code is safe

> M8 adds no new flags. It adds two **checking tools** for the code that reads data the guest controls: **Kani** tries *every possible* input (a proof); the **fuzzer** throws *millions of random* inputs. Both look for crashes, overflows and out-of-bounds reads.



Run the Kani proofs

SYNTAX

```
cargo kani [--harness <NAME>]
```

```
$ cargo install --locked kani-verifier && cargo kani setup   # once, no sudo
$ cargo kani                                  # all 7 proofs, about a minute here
Checking harness devices::virtio::proofs::walk_chain_bounded...
VERIFICATION:- SUCCESSFUL
$ cargo kani --harness blk_parse_no_panic     # just one proof
```

Proof name	It proves that...
`pop_stays_in_ring`	reading the guest's queue never goes outside the queue
`walk_chain_bounded`	following a list of buffers always stops; loops are rejected
`blk_parse_no_panic`	reading a disk request never crashes
`blk_parse_ranges_in_memory`	every accepted buffer lies inside guest RAM
`blk_parse_sector_bounds`	no reads/writes past the end of the disk; no overflow
`net_tx_bounds`	outgoing packets that are too small or too big are rejected
`net_tx_header_zeroed`	the header handed to the host is always all zeros

Run the fuzzer

SYNTAX

```
cargo +nightly fuzz run blk_request <CORPUS_DIR> -- -max_total_time=<SECONDS>
```

```
$ rustup toolchain install nightly && cargo install cargo-fuzz   # once
$ cd fuzz
$ cp -r corpus/blk_request /tmp/corpus       # fuzz a COPY: the fuzzer adds files to it
$ cargo +nightly fuzz run blk_request /tmp/corpus -- -max_total_time=60
```

"Done" with no `crash-...` file in `fuzz/artifacts/` means no problem found.

All

Everything at once

A guest with 4 CPUs, 1 GiB RAM, a disk and the network, with the guard on:

```
$ sudo scripts/setup-tap.sh                # if not done since the last WSL restart
$ cp artifacts/disk.ext4 /tmp/disk.img
$ target/debug/vmm boot \
      --kernel  artifacts/bzImage \
      --initrd  artifacts/initramfs.cpio \
      --mem-mib 1024 \
      --vcpus   4 \
      --disk    /tmp/disk.img \
      --tap     vmtap0
phase1# mkdir -p /mnt && mount /dev/vda /mnt
phase1# ip addr add 172.16.0.2/24 dev eth0 && ip link set eth0 up
phase1# ping -c 1 172.16.0.1 && nproc && cat /mnt/hello.txt
phase1# umount /mnt && reboot -f
```

The full syntax

vmm raw

```
vmm raw --image <FILE> --mode <real|long> [--mem-mib <1-3072>]
```

vmm boot

```
vmm boot --kernel <bzImage> [--initrd <cpio>] [--mem-mib <128-3072>] [--cmdline "<TEXT>"]
         [--vcpus <1-4>] [--disk <FILE> [--disk-ro]] [--tap <IFNAME> [--guest-mac <MAC>]] [-
-no-seccomp]
```

Checking that everything still works

Command	What it checks
`cargo test`	Fast tests (M1, M2, unit tests). No kernel needed.
`cargo test -- --ignored --test-threads=1`	Also the slow tests that boot Linux (M3 to M7). Needs the built kernel, disk and `vmtap0`.
`scripts/verify-phase-1.sh`	Every Phase 1 rule, one PASS/FAIL line each.
`scripts/verify-phase-2.sh`	Every Phase 2 rule, including Phase 1, Kani and 300 s of fuzzing. Shorter fuzz: `FUZZ_SECONDS=30 scripts/verify-phase-2.sh`
`cargo fmt --check && cargo clippy --all-targets -- -D warnings`	Code style and lint rules.

Help

Exit codes

After the VMM ends, `echo $?` shows its exit code.

Code	Meaning
0	All good. `raw` : the program halted. `boot` : the guest rebooted or shut down (also after a guest kernel panic, because `panic=1` reboots).
1	Something broke while running: an unexpected event from the CPU, an internal error, or the VMM was stopped by a signal (`pkill` , closing the terminal). The reason is on stderr.
2	Couldn't start: a bad flag, a missing file, no access to `/dev/kvm` , wrong disk size, missing TAP...
159 / "Bad system call"	The seccomp guard killed the VMM (signal SIGSYS = 31, shell shows 128+31).

When something goes wrong

You see	Do this
`vmm: cannot open KVM device /dev/kvm: ...`	KVM missing or no permission `ls -l /dev/kvm` and that y `kvm` group.
`vmm: cannot read kernel nope: No such file or directory`	Wrong path. Run from the rep build it: `scripts/build-ker`
`error: invalid value '5' for '--vcpus <VCPUS>': 5 is not in 1..=4`	Value out of range. Read the a in the flag tables.
`error: the following required arguments were not provided: --disk <DISK>`	You used `--disk-ro` withou (same for `--guest-mac` with
`vmm: cannot use disk image ...: size is 1000 bytes; it must be a non-zero multiple of 512`	Make the file a proper size, e.g `truncate -s 64M file` , or `scripts/build-disk.sh` .
`vmm: cannot use TAP interface vmtap0: no such network interface`	Run `sudo scripts/setup-t` (needed after every WSL restar
`... it is a multicast address`	The first byte of `--guest-ma` even, e.g. `06:...` .
Boot messages, then `Kernel panic - not syncing: VFS: Unable to mount root fs` , VMM exits 0	You forgot `--initrd artifacts/init`
Nothing appears at all after `vmm boot`	Your `--cmdline` is missing `console=ttyS0` .
Ctrl-C doesn't quit	Correct: it goes to the guest. T `reboot -f` .
Terminal shows no typed text after the VMM ended	Type `reset` and press Enter.
`mount: mounting /dev/vda on /mnt failed`	Did you boot with `--disk` ? F `mkdir -p /mnt` first.
`ping: sendto: Network is unreachable` in guest	Run the two `ip` commands fr 3.

Card

Cheat sheet

From	Host: type this	What it does
M1	`scripts/build-guests.sh`	build the tiny programs
M1	`vmm raw --image guests/build/hello16.bin --mode real`	16-bit hello
M2	`vmm raw --image guests/build/hello64.bin --mode long`	64-bit hello
M3	`scripts/build-kernel.sh` , `scripts/build-initramfs.sh`	build Linux + starter files
M3	`vmm boot --kernel artifacts/bzImage --initrd artifacts/initramfs.cpio`	boot Linux
M3	`... --mem-mib 1024` / `--cmdline "..."`	RAM / kernel settings
M4	`scripts/build-disk.sh` ; `cp artifacts/disk.ext4 /tmp/disk.img`	make a disk copy
M4	`... --disk /tmp/disk.img [--disk-ro]`	attach a disk
M5	`sudo scripts/setup-tap.sh`	create the network cable
M5	`... --tap vmtap0 [--guest-mac 06:..]`	attach the network
M6	`... --vcpus 4`	4 CPUs
M7	`... --no-seccomp`	guard off (debug only)
M8	`cargo kani`	run all proofs
M8	`cargo +nightly fuzz run blk_request <dir>`	fuzz (in `fuzz/`)

`vmm` above is short for `target/debug/vmm` .

From	Guest (`phase1#`): type this	What it does
M3	`reboot -f`	quit the VMM (exit 0)
M3	`uname -r` , `ls /` , `cat /proc/meminfo`	look around
M4	`mkdir -p /mnt && mount /dev/vda /mnt`	open the disk
M4	`sync` , `umount /mnt`	save, then close the disk
M5	`ip addr add 172.16.0.2/24 dev eth0`	give the guest its address
M5	`ip link set eth0 up`	switch the network on
M5	`ping -c 3 172.16.0.1` , `wget -O f http://172.16.0.1:8000/f`	talk to the host, download
M6	`nproc` , `grep LOC /proc/interrupts`	count / check CPUs

● exit 0 = fine ● exit 1 = broke while running ● exit 2 = couldn't start